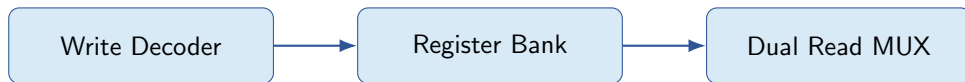


SINGLE-CYCLE PROCESSOR

Part I: Design and Verification of a 32×32 -bit Register File



Tác giả: Phạm Viết Nhật Huy
Chuyên ngành: Thiết kế vi mạch
Ngày hoàn thành: 07/09/2026

Abstract

Báo cáo trình bày phần đầu của bộ xử lý single-cycle: register file 32 thanh ghi, mỗi thanh ghi rộng 32 bit, gồm hai cổng đọc tổ hợp và một cổng ghi đồng bộ. Thanh ghi R0 được nối cứng về 0. Thiết kế được phân rã thành decoder 5-to-32, bank thanh ghi và hai mạng chọn dữ liệu đọc. Môi trường kiểm chứng sử dụng scoreboard, directed tests và 500 giao dịch ngẫu nhiên. Kết quả ModelSim ghi nhận 1074 phép kiểm tra với 0 lỗi.

1 Register File

1.1 Đặc tả

Register file cung cấp đồng thời hai toán hạng nguồn cho datapath và nhận một kết quả từ đường write-back tại cạnh lên của clock. Các yêu cầu thiết kế được tóm tắt trong Bảng 1.

Table 1: Đặc tả register file

Thuộc tính	Giá trị
Kích thước	32 thanh ghi $\times$ 32 bit
Cổng đọc	2 cổng, đọc tổ hợp
Cổng ghi	1 cổng, ghi tại cạnh lên
Reset	Bất đồng bộ, active-high
Thanh ghi R0	Luôn bằng 0, bỏ qua thao tác ghi
Độ rộng địa chỉ	5 bit

Chỉ R1–R31 cần phần tử nhớ, nên số D flip-flop là

$$N_{DFF} = 31 \times 32 = 992.$$

1.2 Microarchitecture

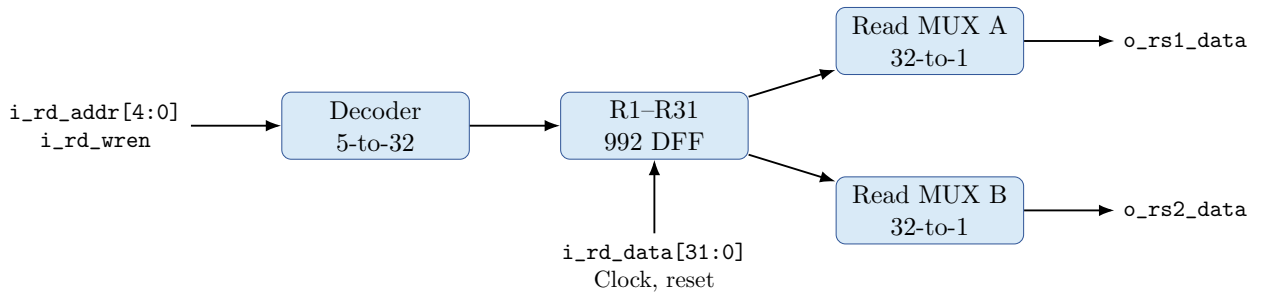


Figure 1: Microarchitecture của register file 2-read/1-write

Đường ghi gồm địa chỉ ghi, decoder và local write-enable. Dữ liệu ghi được broadcast đến toàn bộ bank, nhưng chỉ register có enable bằng 1 cập nhật tại cạnh lên. Decoder chia địa chỉ thành hai trường: hai bit cao chọn một trong bốn nhóm và ba bit thấp chọn một trong tám vị trí. Với nhóm g và vị trí j ,

$$o_dec[8g + j] = i_en \wedge group_sel[g] \wedge index_sel[j].$$

Hai cổng đọc là hai mux độc lập. Vì vậy chúng có thể đọc hai địa chỉ khác nhau trong cùng một chu kỳ. Trước cạnh lên, thao tác đọc cùng địa chỉ ghi trả dữ liệu cũ; sau cạnh lên và độ trễ tổ hợp, đầu ra trả dữ liệu mới.

1.3 Verification methodology

Testbench duy trì một mảng tham chiếu độc lập với DUT. Sau mỗi thao tác ghi hợp lệ, scoreboard cập nhật mô hình và dùng case inequality để so sánh hai cổng đọc, nhờ đó phát hiện cả giá trị X/Z. Test plan gồm reset bất đồng bộ, R1, địa chỉ biên R31, ghi R0, giữ dữ liệu khi write-enable bằng 0, đọc đồng thời, random test và reset sau hoạt động.

Table 2: Các nhóm kiểm thử chính

ID	Mục tiêu	Kỳ vọng
T01	Reset giữa hai cạnh clock	R1–R31 về 0 ngay
T02	Ghi/đọc R1 và R31	Đúng dữ liệu, không ghi nhầm
T03	Thử ghi R0	R0 vẫn bằng 0
T04	Ghi với wren=0	Giữ trạng thái cũ
T05	500 giao dịch ngẫu nhiên	DUT khớp scoreboard
T06	Reset sau hoạt động	Toàn bộ trạng thái hợp lệ về 0

1.4 Kết quả thực nghiệm

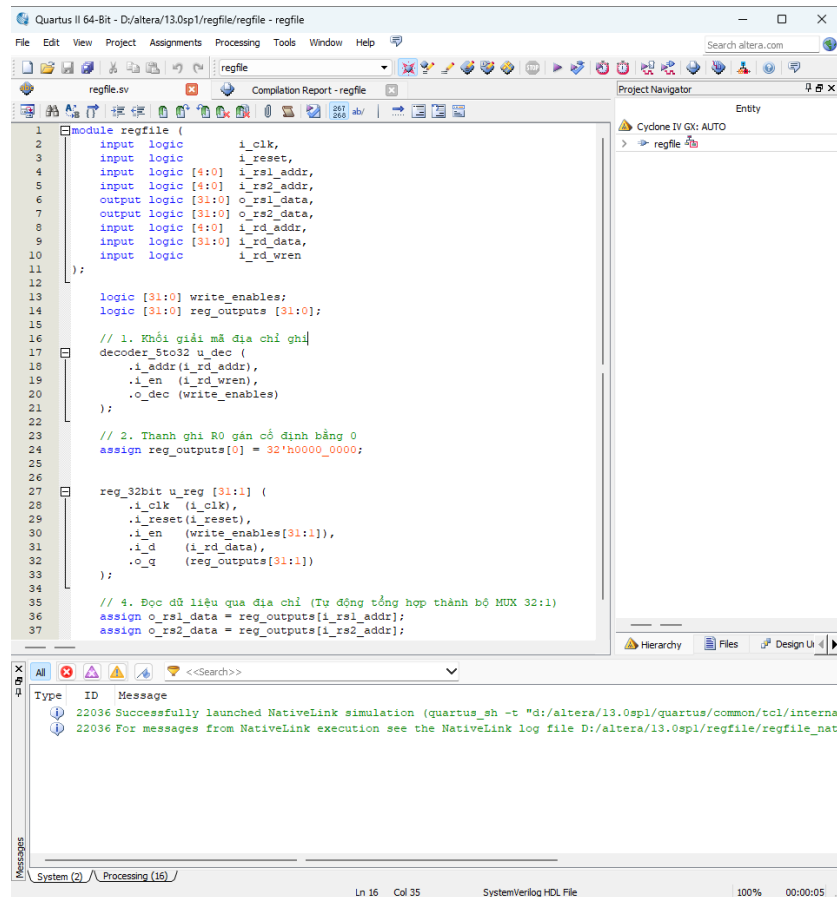


Figure 2: Project register file trong Quartus II 64-bit và NativeLink simulation

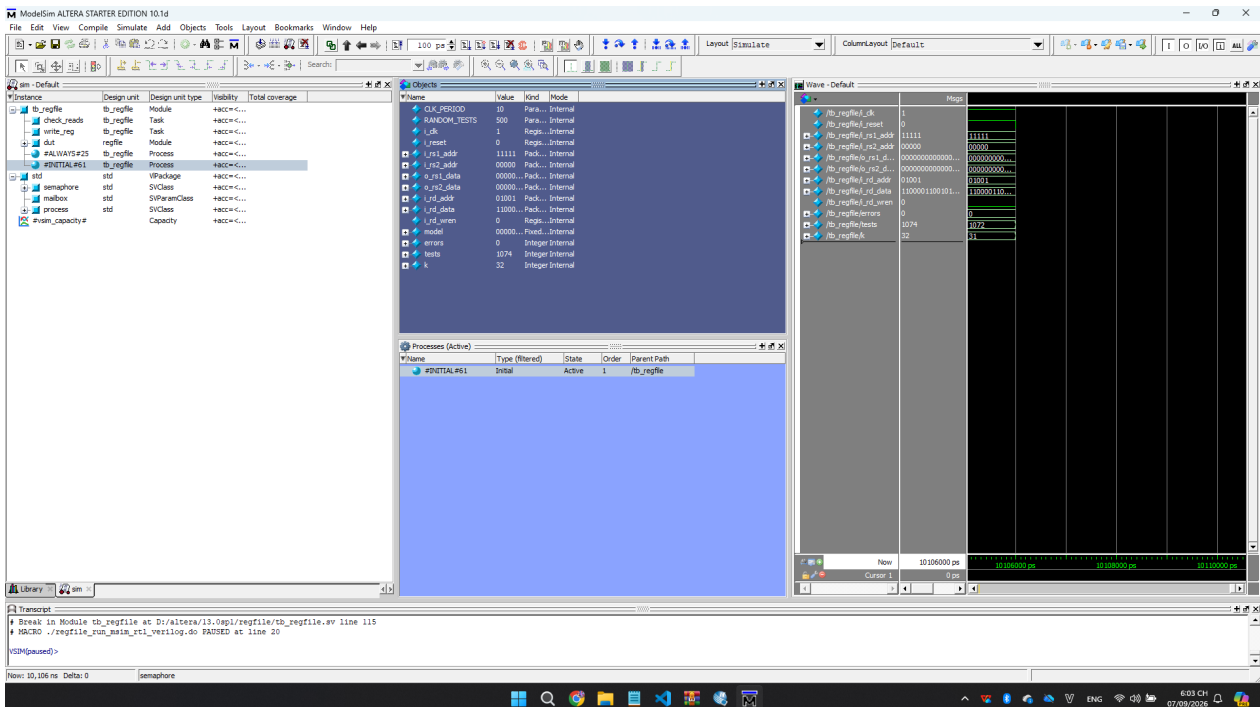


Figure 3: Kết quả ModelSim-Altera: 1074 checks, errors=0

Hình 3 cho thấy testbench dừng tại lệnh `$finish`. Biến `tests=1074` và `errors=0` xác nhận toàn bộ phép so sánh đã thực hiện đều đạt yêu cầu. Việc dừng tại dòng `$finish` là kết thúc mô phỏng, không phải lỗi RTL.

1.5 Nhân xét thiết kế

Phiên bản structural làm rõ decoder, register bank và các cổng đọc, phù hợp để phân tích microarchitecture. Đối với dự án lớn, cùng chức năng thường được mô tả bằng mảng register tham số hóa để giảm mã lặp và tăng khả năng bảo trì. Khi tích hợp vào CPU, đường đọc *register output* $\rightarrow$ *mux* $\rightarrow$ *ALU* có thể trở thành critical path. Mạng clock và reset cũng có fanout 992, cần được xem xét trong synthesis và physical design.

2 Hướng phát triển Single-Cycle Processor

Register file sẽ được kết nối với instruction decoder, immediate generator, ALU, data memory và write-back mux. Các bước tiếp theo gồm thiết kế ALU 32 bit, program counter, control unit, instruction/data memory interface, sau đó kiểm chứng các lệnh RV32I cơ bản ở cấp độ processor.

3 Kết luận

Register file đạt chức năng 2-read/1-write, bảo vệ R0 và vượt qua 1074 phép kiểm tra tự động. Thiết kế chứng minh được mối liên hệ giữa specification, microarchitecture, RTL structural và scoreboard-based verification, tạo nền tảng cho datapath của bộ xử lý single-cycle.

A Mã nguồn SystemVerilog

A.1 D flip-flop có enable

```
// D Flip-Flop 1-bit có Reset bất đồng bộ và Write Enable
module dff_en (
    input logic i_clk,
    input logic i_reset,
    input logic i_en,
    input logic i_d,
    output logic o_q
);
    always_ff @(posedge i_clk or posedge i_reset) begin
        if (i_reset)
            o_q <= 1'b0;
        else if (i_en)
            o_q <= i_d;
        end
    end
endmodule
```

A.2 Thanh ghi 32 bit

```
module reg_32bit (
    input logic i_clk,
    input logic i_reset,
    input logic i_en,
    input logic [31:0] i_d,
    output logic [31:0] o_q
);
    // Explicit generate is portable across older Quartus/ModelSim versions.
    genvar bit_idx;
    generate
        for (bit_idx = 0; bit_idx < 32; bit_idx = bit_idx + 1) begin : gen_dff
            dff_en u_dff (
                .i_clk (i_clk),
                .i_reset(i_reset),
                .i_en (i_en),
                .i_d (i_d[bit_idx]),
                .o_q (o_q[bit_idx])
            );
        end
    endgenerate
endmodule
```

A.3 Decoder 5-to-32 cấu trúc

```
module decoder_5to32 (
    input logic [4:0] i_addr,
    input logic i_en,
    output wire [31:0] o_dec
);
    wire [3:0] group_sel;
    wire [7:0] index_sel;

    decoder_2to4 u_group (.i_a(i_addr[4:3]), .i_en(i_en), .o_y(group_sel));
    decoder_3to8 u_index (.i_a(i_addr[2:0]), .i_en(i_en), .o_y(index_sel));

    and (o_dec[0],group_sel[0],index_sel[0]); and (o_dec[1],group_sel[0],index_sel[1]);
    and (o_dec[2],group_sel[0],index_sel[2]); and (o_dec[3],group_sel[0],index_sel[3]);
    and (o_dec[4],group_sel[0],index_sel[4]); and (o_dec[5],group_sel[0],index_sel[5]);
    and (o_dec[6],group_sel[0],index_sel[6]); and (o_dec[7],group_sel[0],index_sel[7]);
    and (o_dec[8],group_sel[1],index_sel[0]); and (o_dec[9],group_sel[1],index_sel[1]);
    and (o_dec[10],group_sel[1],index_sel[2]); and (o_dec[11],group_sel[1],index_sel[3]);
    and (o_dec[12],group_sel[1],index_sel[4]); and (o_dec[13],group_sel[1],index_sel[5]);
    and (o_dec[14],group_sel[1],index_sel[6]); and (o_dec[15],group_sel[1],index_sel[7]);
    and (o_dec[16],group_sel[2],index_sel[0]); and (o_dec[17],group_sel[2],index_sel[1]);
    and (o_dec[18],group_sel[2],index_sel[2]); and (o_dec[19],group_sel[2],index_sel[3]);
    and (o_dec[20],group_sel[2],index_sel[4]); and (o_dec[21],group_sel[2],index_sel[5]);
    and (o_dec[22],group_sel[2],index_sel[6]); and (o_dec[23],group_sel[2],index_sel[7]);
    and (o_dec[24],group_sel[3],index_sel[0]); and (o_dec[25],group_sel[3],index_sel[1]);
    and (o_dec[26],group_sel[3],index_sel[2]); and (o_dec[27],group_sel[3],index_sel[3]);
    and (o_dec[28],group_sel[3],index_sel[4]); and (o_dec[29],group_sel[3],index_sel[5]);
    and (o_dec[30],group_sel[3],index_sel[6]); and (o_dec[31],group_sel[3],index_sel[7]);
endmodule

module decoder_2to4 (input logic [1:0] i_a, input logic i_en, output wire [3:0] o_y);
    wire na1, na0;
    not (na1,i_a[1]); not (na0,i_a[0]);
    and (o_y[0],i_en,na1,na0); and (o_y[1],i_en,na1,i_a[0]);
    and (o_y[2],i_en,i_a[1],na0); and (o_y[3],i_en,i_a[1],i_a[0]);
endmodule

module decoder_3to8 (input logic [2:0] i_a, input logic i_en, output wire [7:0] o_y);
    wire na2, na1, na0;
```

```

    not (na2,i_a[2]); not (na1,i_a[1]); not (na0,i_a[0]);
    and (o_y[0],i_en,na2,na1,na0); and (o_y[1],i_en,na2,na1,i_a[0]);
    and (o_y[2],i_en,na2,i_a[1],na0); and (o_y[3],i_en,na2,i_a[1],i_a[0]);
    and (o_y[4],i_en,i_a[2],na1,na0); and (o_y[5],i_en,i_a[2],na1,i_a[0]);
    and (o_y[6],i_en,i_a[2],i_a[1],na0); and (o_y[7],i_en,i_a[2],i_a[1],i_a[0]);
endmodule

```

A.4 Top-level register file

```

module regfile (
    input logic i_clk,
    input logic i_reset,
    input logic [4:0] i_rs1_addr,
    input logic [4:0] i_rs2_addr,
    output logic [31:0] o_rs1_data,
    output logic [31:0] o_rs2_data,
    input logic [4:0] i_rd_addr,
    input logic [31:0] i_rd_data,
    input logic i_rd_wren
);

    logic [31:0] write_enables;
    logic [31:0] reg_outputs [0:31];

    // 1. Khỏi giải mã địa chỉ ghi
    decoder_5to32 u_dec (
        .i_addr(i_rd_addr),
        .i_en (i_rd_wren),
        .o_dec (write_enables)
    );

    // R0 is hardwired to zero; writes to address 0 have no state element.
    assign reg_outputs[0] = 32'h0000_0000;

    genvar reg_idx;
    generate
        for (reg_idx = 1; reg_idx < 32; reg_idx = reg_idx + 1) begin : gen_reg
            reg_32bit u_reg (
                .i_clk (i_clk),
                .i_reset(i_reset),
                .i_en (write_enables[reg_idx]),
                .i_d (i_rd_data),
                .o_q (reg_outputs[reg_idx])
            );
        end
    endgenerate

    // 4. Đọc dữ liệu qua địa chỉ (Tự động tổng hợp thành bộ MUX 32:1)
    assign o_rs1_data = reg_outputs[i_rs1_addr];
    assign o_rs2_data = reg_outputs[i_rs2_addr];

endmodule

```

A.5 Self-checking testbench

```

`timescale 1ns/1ps

module tb_regfile;
    localparam integer CLK_PERIOD = 10;
    localparam integer RANDOM_TESTS = 500;

    logic i_clk;
    logic i_reset;
    logic [4:0] i_rs1_addr;
    logic [4:0] i_rs2_addr;
    logic [31:0] o_rs1_data;
    logic [31:0] o_rs2_data;
    logic [4:0] i_rd_addr;
    logic [31:0] i_rd_data;
    logic i_rd_wren;

    logic [31:0] model [0:31];
    integer errors;
    integer tests;
    integer k;

    regfile dut (.*);

    initial i_clk = 1'b0;
    always #(CLK_PERIOD/2) i_clk = ~i_clk;

    task automatic check_reads(input [4:0] a1, input [4:0] a2);
        begin
            i_rs1_addr = a1;
            i_rs2_addr = a2;
            #1;
            tests = tests + 2;
        end
    endtask
endmodule

```

```

        if (o_rs1_data != model[a1]) begin
            $error("RS1 mismatch: addr=%0d expected=%08h actual=%08h",
                a1, model[a1], o_rs1_data);
            errors = errors + 1;
        end
        if (o_rs2_data != model[a2]) begin
            $error("RS2 mismatch: addr=%0d expected=%08h actual=%08h",
                a2, model[a2], o_rs2_data);
            errors = errors + 1;
        end
    end
endtask

task automatic write_reg(input [4:0] addr, input [31:0] data);
begin
    @(negedge i_clk);
    i_rd_addr = addr;
    i_rd_data = data;
    i_rd_wren = 1'b1;
    @(posedge i_clk);
    #1;
    if (addr != 0)
        model[addr] = data;
    @(negedge i_clk);
    i_rd_wren = 1'b0;
end
endtask

initial begin
    errors = 0;
    tests = 0;
    i_reset = 1'b0;
    i_rs1_addr = 5'd0;
    i_rs2_addr = 5'd0;
    i_rd_addr = 5'd0;
    i_rd_data = 32'd0;
    i_rd_wren = 1'b0;
    for (k = 0; k < 32; k = k + 1)
        model[k] = 32'd0;

    // Asynchronous reset must clear R1..R31 without a clock edge.
    #2 i_reset = 1'b1;
    #1 check_reads(5'd0, 5'd31);
    #2 i_reset = 1'b0;

    // Directed corner cases.
    write_reg(5'd1, 32'h1234_5678);
    check_reads(5'd1, 5'd0);
    write_reg(5'd31, 32'hDEAD_BEEF);
    check_reads(5'd31, 5'd1);

    // R0 must ignore writes.
    write_reg(5'd0, 32'hFFFF_FFFF);
    check_reads(5'd0, 5'd31);

    // Disabled write must preserve the previous value.
    @(negedge i_clk);
    i_rd_addr = 5'd1;
    i_rd_data = 32'hAAAA_5555;
    i_rd_wren = 1'b0;
    @(posedge i_clk);
    #1 check_reads(5'd1, 5'd31);

    // Randomized writes and simultaneous dual-port reads.
    for (k = 0; k < RANDOM_TESTS; k = k + 1) begin
        write_reg($urandom_range(0,31), $urandom);
        check_reads($urandom_range(0,31), $urandom_range(0,31));
    end

    // Reset after activity.
    #2 i_reset = 1'b1;
    for (k = 0; k < 32; k = k + 1)
        model[k] = 32'd0;
    #1;
    for (k = 0; k < 32; k = k + 1)
        check_reads(k[4:0], (31-k));
    i_reset = 1'b0;

    if (errors == 0)
        $display("PASS: %0d checks completed with no errors", tests);
    else
        $fatal(1, "FAIL: %0d of %0d checks failed", errors, tests);
    $finish;
end
endmodule

```